

CerbaSeal

Core · v0.1.0

System Architecture & Technical Reference Manual

The authoritative technical reference for CerbaSeal-Core — the single deterministic enforcement gate that makes it technically impossible to execute a consequential AI-assisted action without satisfying every governance requirement. Covers invariant framework, domain type system, policy layer, audit chain, HTTP API, deployment, and partner delivery.

VERSION

0.1.0

STATUS

GENERALLY AVAILABLE

DATE

June 2026

AUTHOR

Jesse Lamont

Table of Contents

1	Executive Summary	3
2	System Overview	4
3	Core Architecture	5
3.1	Enforcement Gate — ExecutionGateService	5
3.2	Invariant Assertion Functions	6
3.3	Policy Evaluation Stages	7
3.4	Gate-Issued Result Registry	7
3.5	Decision Flow Summary	8
4	Invariant Framework	9
5	Domain Type Reference	10
5.1	GovernedRequest	10
5.2	DecisionProposal, PolicyPackRef, ProvenanceRef	11
5.3	ApprovalArtifact	11
5.4	ControlStatus & TrustState	12
5.5	GateResult	12
5.6	DecisionEnvelope & DecisionTrace	12
5.7	ReleaseAuthorization & BlockedActionRecord	13
5.8	Discriminated Union Types	13
6	Reason Code Reference	14
7	Configuration System	15
7.1	CerbaSealConfig & GateConfig	15
7.2	CerbaSealPolicy	16
7.3	WorkflowRule & PolicyActionBehavior	17
8	Audit & Evidence Chain	17
8.1	IAuditLogService Interface	17
8.2	AppendOnlyLogService (In-Memory)	18
8.3	FileBackedAppendOnlyLogService (Persistent)	18
8.4	EvidenceBundleService	19
8.5	Audit Event Types & AuditLogEntry	19
9	HTTP API Reference	20
10	Deployment Architecture	21
10.1	Path A — Docker Compose	21
10.2	Path B — Node.js Direct	22
11	Test Coverage	23
12	Partner Delivery Framework	24
13	Security Properties & Threat Model	25

Executive Summary

CerbaSeal-Core is a deterministic execution governance library for organisations that use AI-assisted decision making in regulated workflows. It solves a structural problem: the path from "AI recommends" to "action executes" is almost always unenforced. CerbaSeal makes that bypass technically impossible.

The enforcement gate evaluates every GovernedRequest against 12 hard invariants before issuing a ReleaseAuthorization. The invariants are unconditional — no configuration, no policy, no caller can relax them. A client-authored policy layer (CerbaSealPolicy) adds client-specific restrictions on top of the invariants, never below them.

Key Guarantees

Single enforcement point

Every consequential action must pass through ExecutionGateService.evaluate(). No secondary API, no override path, no bypass.

Deterministic decisions

evaluate() is a pure, synchronous function. The same request evaluated twice always produces the same decision. No network calls, no randomness, no side effects.

Gate-issued result integrity

A module-private WeakSet (_gateIssuedResults) tracks every GateResult issued by evaluate(). EvidenceBundleService rejects any result not in that registry.

Immutable evidence

DecisionEnvelope carries immutable: true as a literal type. Envelopes are sealed at issuance and cannot be mutated through the public API.

Tamper-evident audit log

SHA-256 forward chaining — each AuditLogEntry hash includes the previous entry hash. verifyChain() detects any historical modification instantly.

AI non-authoritativeness

INV-05 is checked on every request. An actor with actorAuthorityClass "ai" cannot authorise its own proposals. An AI-sourced proposal (proposalSourceKind "ai")

submitted by an "ai" actor is rejected unconditionally.

Policy is additive only

CerbaSealPolicy can add approval requirements, block actions, and remap actors. It cannot grant exemptions to core invariants.

What CerbaSeal Is Not

- CerbaSeal is not an AI model, workflow engine, or case management system.
- It does not make decisions — it evaluates whether a proposed decision is authorised to execute.
- It is not a hosted service — it is a library deployed inside the client's own infrastructure.
- It is not a replacement for compliance tooling — it provides the enforcement foundation compliance tooling audits.

v0.1.0 Scope

Drop 01 deliverables: enforcement gate (ExecutionGateService), in-memory and file-backed audit log, evidence bundle and export, configuration and policy loaders, HTTP deployment server, diagnostic report service, and the Partner Delivery Kit (7 documents, 3 certification levels). HMAC signing over DecisionEnvelope is available in source but not yet exposed as a first-class API. Third-party security review is planned for v0.2.0.

CerbaSeal is structured as seven discrete layers. Each layer has a specific job. The request flows upward through the stack; evidence and audit records flow downward and are captured at every layer.

[illegible]

Layer	Component	Job	Partner Integration Point
1 — Request	GovernedRequest	Structured input carrying actor, action, provenance, trust state, and control status	Client AI system constructs the request object from its output and session context
2 — Trust	TrustState	Assert the requesting actor is currently trusted (no expired session, no revoked token)	Client IdP sets trusted: true/false; trustStateId provides audit traceability
3 — Provenan	ProvenanceRef	Trace the AI model version, rule set version, and source hash that generated the proposal	Client AI pipeline exposes modelVersion, ruleSetVersion, and sourceHash
4 — Gate	ExecutionGateService	Evaluate all 12 invariants + policy. Issue GateResult with ALLOW / HOLD / REJECT.	Partners instantiate the gate with config + policy; never modify gate source
5 — Policy	CerbaSealPolicy	Apply client-specific approval chains, actor mappings, workflow rules, and action policies	Partners author cerbaseal.policy.json per client workflow during pilot
6 — Audit	AppendOnlyLogService	Append-only JSONL log with SHA-256 forward chain. verifyChain() detects any tampering.	Partners configure LOG_PATH and persistence volume; log path in env var
7 — Evidence	EvidenceBundleService	Assemble full proof bundle (request + envelope + chain). Drive ExportManifest for client handover.	Partners run pnpm export:proof at pilot close; pnpm verify:proof confirms integrity

Decision Lifecycle

Stage	Who	What Happens
Construct	Client integration	Builds GovernedRequest from AI output, identity context, provenance metadata, and control state
Submit	Client integration	Calls gate.evaluate(request) — synchronous, pure, no await
Schema check	Gate (INV-11,12)	assertRequestShape() + assertKnownActionClass() + assertActorAuthorityClass() + proposal/action match
Policy stage 1	Gate	Actor mapping: client role name translated to canonical authority class via actorMappings
Policy stage 2	Gate	"blocked" action policies cause immediate REJECT before any other invariant runs
Policy stage 3	Gate	Compute effective approvalRequired from workflowRules + approvalChains + actionPolicies
Invariant checks	Gate (INV-01..10)	assertPolicyPack, assertProvenance, assertLoggingReady, assertProposalBoundary, assertProhibitedUse, assertControlStatus, assertTrustState, assertApprovalState
Policy stage 4	Gate	Approval chain authority enforcement: approver class must be in the configured chain for this workflow
Decision	Gate	ALLOW != ReleaseAuthorization issued. HOLD/REJECT != BlockedActionRecord. DecisionEnvelope always produced.
Registration	Gate	GateResult added to _gateIssuedResults WeakSet before return
Audit	Caller / Evidence	Caller appends REQUEST_EVALUATED to IAuditLogService; EvidenceBundleService assembles bundle
Export	Partner	pnpm export:proof generates ExportManifest with chain hashes and proof snapshot

3

Core Architecture

3.1 Enforcement Gate — ExecutionGateService

ExecutionGateService is the only enforcement point. It is instantiated once per process at startup and its evaluate() method is the sole public mutation-bearing operation. The constructor accepts an optional GateConfig (which is a union of CerbaSealConfig | ExecutionGateConfig) and an optional CerbaSealPolicy.

ExecutionGateService — instantiation and evaluation

```
import { ExecutionGateService } from "../src/services/execution/execution-gate-service.js";
import { loadCerbaSealConfig } from "../src/config/cerbaseal-config.js";
import { loadCerbaSealPolicy } from "../src/config/cerbaseal-policy.js";

// From files (recommended for deployment):
const gate = new ExecutionGateService(loadCerbaSealConfig(), loadCerbaSealPolicy());

// Inline (useful in tests):
const gate = new ExecutionGateService(
  { additionalAuthorityClasses: ["risk_officer"] },
  { actorMappings: { "Head of Risk": "manager" } }
);

const result = gate.evaluate(request); // GateResult — synchronous, pure
```

Constructor parameter	Type	Default	Purpose
config	GateConfig (CerbaSealConfig ExecutionGateConfig)	{}	Extends allowed authority class set. CerbaSealConfig uses
policy	CerbaSealPolicy undefined	undefined	Supplies actor mappings, approval chains, workflow rules, and action policies. When undefined: gate operates on core invariants only.

Private field:

allowedAuthorityClasses

ReadonlySet<string> built at construction from CORE_AUTHORITY_CLASSES + config extensions + actorMappings keys from policy. Callers cannot access or modify it.

Private field: policy

CerbaSealPolicy | undefined — stored for use in all four policy evaluation stages during evaluate().

3.2 Invariant Assertion Functions

Each invariant is implemented as a standalone function that pushes its InvariantCode onto the checkedInvariants array, then throws CerbaSealError on violation. CerbaSealError always carries { invariant, reasonCode, finalState }. The catch block in evaluate() converts any CerbaSealError into a controlled GateResult; non-CerbaSealError

exceptions are caught by a second fallback that also produces a REJECT result.

Function	Invariant(s)	Key checks performed
assertRequestShape()	INV-11	requestId non-empty, jurisdiction non-empty, createdAt non-empty, proposal.reasonCodes.length > 0
assertActorAuthorityClass()	INV-11	actorAuthorityClass is in allowedAuthorityClasses set (after actor mapping resolution)
assertKnownActionClass()	INV-11	proposedActionClass and proposal.requestedActionClass both in ALLOWED ACTION CLASSES set
proposal/action match	INV-12	proposedActionClass === proposal.requestedActionClass (checked inline in evaluate())
assertPolicyPack()	INV-01	policyPackRef !== null
assertProvenance()	INV-02	provenanceRef !== null; modelVersion, ruleSetVersion, sourceHash all non-empty
assertLoggingReady()	INV-04	loggingReady === true
assertProposalBoundary()	INV-05	proposal.authorityBearing === false AND NOT (proposalSourceKind "ai" AND actorAuthorityClass "ai")
assertProhibitedUse()	INV-10	prohibitedUse === false
assertControlStatus()	INV-08	If sensitive: criticalControlsValid === true AND stale === false
assertTrustState()	INV-09	trustState.trusted === true
assertApprovalState()	INV-03	See §3.3 — complex multi-check function

assertApprovalState() — Detail

This is the most complex assertion. It first computes `effectiveApprovalRequired = request.approvalRequired OR WORKFLOWS_REQUIRING_APPROVAL.has(workflowClass)`. The hardcoded set `WORKFLOWS_REQUIRING_APPROVAL = { "fraud_triage" }` — the caller cannot opt out of approval for `fraud_triage` by setting `approvalRequired: false`.

Check	Failure	Error details
<code>approvalArtifact === null (when effective approval required)</code>	HOLD	REQUIRED_APPROVAL_MISSING — request is holdable (not terminal reject) awaiting human review
<code>approvalArtifact.forRequestId !== request.requestId</code>	REJECT	INVALID_APPROVAL_AUTHORITY — prevents reuse of a valid approval across different requests
<code>approvedAt</code> timestamp is invalid ISO or predates <code>createdAt</code>	REJECT	INVALID_APPROVAL_AUTHORITY or MALFORMED_REQUEST — forged or replayed
<code>approverAuthorityClass</code> not in <code>HumanAuthorityClass</code> set	REJECT	INVALID_APPROVAL_AUTHORITY — ai, system not valid approvers at this level
<code>privilegedAuthSatisfied === false</code>	REJECT	PRIVILEGED_AUTH_NOT_SATISFIED — MFA or step-up auth was not completed
<code>immutableSignature</code> empty or whitespace	REJECT	APPROVAL_SIGNATURE_MISSING — signature field required (content not validated in v0.1.0)

3.3 Policy Evaluation Stages

Policy rules are evaluated in four stages interspersed with the invariant checks. They run only on the subset of requests that reach each stage (earlier failures short-circuit the evaluation).

Stage	When	What it does
Stage 1 — Actor mapping	After <code>assertRequestShape</code>	Translates client role names to canonical authority classes via <code>policy.actorMappings</code> . Resolved class is used for all downstream invariant
Stage 2 — Blocked action policies	After INV-12 match check	"blocked" action policies produce immediate REJECT before provenance, logging, or approval checks run. Fast-fail for categorically forbidden actions.
Stage 3 — Effective approval	Before <code>assertPolicyPack</code>	Computes effective <code>approvalRequired</code> from <code>workflowRules</code> + <code>approvalChains</code> + "requires_approval" action policies. If policy requires approval and request
Stage 4 — Approval chain authority	After all invariants pass	If <code>policy.approvalChains</code> defines a chain for this workflow, the approver's class must be in that list. Narrows valid approvers beyond the core <code>HumanAuthorityClass</code> check.

3.4 Gate-Issued Result Registry

A module-private `WeakSet` — `const _gateIssuedResults = new WeakSet<object>()` — is defined at module scope in `execution-gate-service.ts`. Every `GateResult` returned by `evaluate()` is registered: `_gateIssuedResults.add(result)` before return. The exported `assertIsGateIssued(result)` function checks membership and throws `CerbaSealError(INV-06, MALFORMED_REQUEST, REJECT)` if the result is absent.

The `WeakSet` is not exported. It is not accessible via any public API. A caller cannot register their own `GateResult`. This is the structural guarantee behind INV-06 (NO_BYPASS_OF_EXECUTION_GATE).

3.5 Decision Flow Summary

Complete evaluate() decision tree

```
gate.evaluate(request)
%
% % assertRequestShape / assertActorAuthorityClass / assertKnownActionClass fails
%   !' throw CerbaSealError !' REJECT (caught, controlled GateResult)
%
% % Proposal/action mismatch (INV-12) !' REJECT
%
% % Policy stage 2: action "blocked" !' REJECT
%
% % assertPolicyPack null (INV-01) !' REJECT
% % assertProvenance null/incomplete (INV-02) !' REJECT
% % assertLoggingReady false (INV-04) !' REJECT
% % assertProposalBoundary (INV-05) !' REJECT
% % assertProhibitedUse true (INV-10) !' REJECT
% % assertControlStatus stale/invalid (INV-08) !' REJECT
% % assertTrustState untrusted (INV-09) !' REJECT
%
% % assertApprovalState approval missing !' HOLD
% % assertApprovalState forRequestId mismatch !' REJECT
% % assertApprovalState timestamp invalid/predates !' REJECT
% % assertApprovalState invalid approver class !' REJECT
% % assertApprovalState privilegedAuthSatisfied false !' REJECT
% % assertApprovalState signature empty !' REJECT
%
% % Policy stage 4: approver not in approval chain !' REJECT
%
% % All checks pass !' ALLOW
    ReleaseAuthorization issued. Result registered in _gateIssuedResults.
```


4

Invariant Framework

Invariant contract: All 12 invariants are unconditional. They are evaluated before policy rules on every request. Policy cannot suppress or relax any invariant. This is the foundational guarantee CerbaSeal provides to compliance teams and auditors.

Code	Constant Name	What It Enforces	State on Failure
INV-01	NO_POLICY_PACK_NO_EXECUTION	policyPackRef must be non-null. No policy reference means no execution path exists in CerbaSeal.	REJECT
INV-02	NO_PROVENANCE_NO_ACTION	provenanceRef must be non-null with all three sub-fields (modelVersion, ruleSetVersion, sourceHash) non-empty.	REJECT
INV-03	NO_REQUIRED_APPROVAL_NO_RELEASE	When approval is effectively required (caller flag OR WORKFLOWS_REQUIRING_APPROVAL OR policy), a valid	HOLD or REJECT
INV-04	NO_LOGGING_NO_EXECUTION	loggingReady must be true. The caller asserts an audit log is ready to receive the outcome before the gate issues a release.	REJECT
INV-05	AI_NON_AUTHORITATIVE	proposal.authorityBearing must be false. An AI-sourced proposal (proposalSourceKind "ai") submitted by an "ai" actor is	REJECT
INV-06	NO_BYPASS_OF_EXECUTION_GATE	GateResult objects used to assemble evidence must be in the _gateIssuedResults WeakSet. Hand-constructed or externally-	REJECT
INV-07	IMMUTABLE_DECISION_ENVELOPE	DecisionEnvelope.immutable is a literal type true, sealed at issuance. The type system and WeakSet registration together	REJECT
INV-08	STALE_CONTROLS_BLOCK_SENSITIVE_RELEASE	For requests with sensitive: true, controlStatus.criticalControlsValid must be true AND	REJECT
INV-09	TRUST_STATE_REQUIRED	trustState.trusted must be true. An untrusted actor (expired session, revoked token) is rejected unconditionally — no policy	REJECT
INV-10	PROHIBITED_USE_MUST_BLOCK	prohibitedUse: true triggers REJECT before any other checks after schema validation. No configuration, policy, or approval can	REJECT
INV-11	REQUEST_SCHEMA_AND_ACTION_CLASS_VALID	requestId/jurisdiction/createdAt non-empty, proposal.reasonCodes non-empty, actorAuthorityClass	REJECT (throw)
INV-12	PROPOSAL_AND_REQUEST_ACTION_MUST_MATCH	proposedActionClass must exactly equal proposal.requestedActionClass. A proposal for a different action than the request declares is structurally invalid.	REJECT

Domain Type Reference

5.1 GovernedRequest — All 17 Fields

GovernedRequest is the sole input to `gate.evaluate()`. Every field carries a specific governance meaning evaluated by at least one invariant or policy rule. There is no purely informational field.

Field	Type	Required by Invariant / Policy
requestId	string	INV-11: must be non-empty after trim. Correlates audit log, evidence bundle, and release authorization.
workflowClass	WorkflowClass	Core type. Determines approval chain lookup and WORKFLOWS REQUIRING APPROVAL membership check.
jurisdiction	string	INV-11: must be non-empty. Informational for evidence; extensible for future policy-level jurisdiction rules.
actorId	string	Included in audit log payload. No invariant check beyond schema; used for evidence and diagnostic reports.
actorAuthorityClass	AuthorityClass	INV-11: must be in allowedAuthorityClasses. INV-05: must not be "ai" (after actor manning resolution).
proposedActionClass	UnknownableActionClass	INV-11: must be in ALLOWED_ACTION_CLASSES. INV-12: must match proposal.requestedActionClass.
proposal	DecisionProposal	INV-05: proposal.authorityBearing must be false. INV-12: proposal.requestedActionClass must match proposedActionClass.
sensitive	boolean	INV-08: if true, controlStatus.criticalControlsValid must be true and controlStatus.stale must be false.
prohibitedUse	boolean	INV-10: if true, immediate REJECT unconditionally.
policyPackRef	PolicyPackRef null	INV-01: must be non-null.
provenanceRef	ProvenanceRef null	INV-02: must be non-null with all sub-fields non-empty.
approvalRequired	boolean	INV-03: when true (or overridden by policy), approvalArtifact must pass all checks.
approvalArtifact	ApprovalArtifact null	INV-03: required when effectiveApprovalRequired. Subject to 6 sub-checks (see §3.2).
loggingReady	boolean	INV-04: must be true. Caller asserts the audit log service is ready.
controlStatus	ControlStatus	INV-08: { criticalControlsValid, stale, verificationRunId }. Checked for sensitive requests.
trustState	TrustState	INV-09: { trusted, trustStateId }. trusted must be true unconditionally.
createdAt	string (ISO 8601)	INV-11: must be non-empty. INV-03: approvalArtifact.approvedAt must not predate this value.

5.2 DecisionProposal, PolicyPackRef, ProvenanceRef

Type	Fields	Notes
DecisionProposal	proposalSourceKind: "ai" "deterministic_rule" authorityBearing: boolean requestedActionClass: UnknownableActionClass	INV-05 checks authorityBearing (must be false) and the combination of proposalSourceKind + actorAuthorityClass. reasonCodes must be non-
PolicyPackRef	id: string version: string	INV-01: object must be non-null. Individual field content is not validated by the gate — the policy pack is identified but not fetched at evaluation time.
ProvenanceRef	modelVersion: string ruleSetVersion: string sourceHash: string	INV-02: object must be non-null AND all three fields must be non-empty after trim. A partially populated ProvenanceRef fails the invariant.

5.3 ApprovalArtifact — All 7 Fields

Field	Type	Invariant Check
approvalId	string	Identifies this specific approval record. Stored in evidence for audit trail.
approverId	string	Identity of the approving human. Stored in evidence.
forRequestId	string	INV-03: must exactly equal GovernedRequest.requestId. Prevents approval reuse across requests.
approverAuthorityClass	HumanAuthorityClass	INV-03: must be one of analyst reviewer manager compliance officer. "ai" and "system" cannot approve.
privilegedAuthSatisfied	boolean	INV-03: must be true. Represents completion of step-up authentication (MFA, privileged session, etc.).
immutableSignature	string	INV-03: must be non-empty after trim. Content is not cryptographically validated in v0.1.0 (HMAC planned for v0.2.0).
approvedAt	string (ISO 8601)	INV-03: must be a valid ISO date AND must not predate GovernedRequest.createdAt. Prevents replayed artifacts.

5.4 ControlStatus & TrustState

Type	Field	Type	Invariant / Notes
ControlStatus	criticalControlsValid	boolean	INV-08: must be true for sensitive requests. Client control system asserts all mandatory controls are active.
ControlStatus	stale	boolean	INV-08: must be false for sensitive requests. A stale control status is as dangerous as an invalid one.
ControlStatus	verificationRunId	string	Identifies the control verification run. Informational for audit traceability — not checked by gate.
TrustState	trusted	boolean	INV-09: must be true unconditionally. Single false value immediately rejects the request.
TrustState	trustStateId	string	Identifies the trust evaluation event (session token hash, IdP assertion ID, etc.). For audit traceability.

5.5 GateResult

GateResult is the return value of `gate.evaluate()`. All three fields are always present. `releaseAuthorization` and `blockedActionRecord` are mutually exclusive.

GateResult interface (src/domain/types/core.ts)

```
interface GateResult {
  decisionEnvelope: DecisionEnvelope; // always present; immutable: true
  releaseAuthorization: ReleaseAuthorization | null; // non-null iff finalState === "ALLOW"
  blockedActionRecord: BlockedActionRecord | null; // non-null iff finalState === "HOLD" | "REJECT"
}
```

5.6 DecisionEnvelope & DecisionTrace

Field	Type	Source / Notes
envelopeId	string	Deterministic: "env_" + requestId. Stable identifier for this decision record
requestId	string	Echoes GovernedRequest.requestId. Primary correlation key for audit log and evidence
workflowClass	WorkflowClass	Echoes the resolved workflowClass from the (possibly mapped) request
finalState	"ALLOW" "HOLD" "REJECT"	Terminal gate decision.
permittedActionClass	ActionClass null	Non-null only when finalState === "ALLOW". The specific validated action class
humanApprovalRequired	boolean	Reflects request.approvalRequired (the caller-supplied flag, not the effective value)
humanApprovalPresent	boolean	True when approvalArtifact !== null on the request.
proposalSourceKind	"ai" "deterministic_rule"	Copied from proposal.proposalSourceKind for evidence classification.
immutable	true	Literal type true. TypeScript enforces this at compile time; runtime behavior enforces it structurally
evidenceBundled	string	Deterministic: "evidence_" + requestId. Cross-references the EvidenceBundle
trace	DecisionTrace	{ checkedInvariants: InvariantCode[], reasonCodes: ReasonCode[], evaluatedAt: string }
issuedAt	string (ISO)	Timestamp of gate evaluation (nowIso()) call at the end of the happy path).

5.7 ReleaseAuthorization & BlockedActionRecord


```

interface ReleaseAuthorization {
  releaseAuthorizationId: string; // "release_" + requestId
  requestId: string;
  envelopeId: string;
  actionClass: ActionClass; // the validated, permitted action
  releasedAt: string; // ISO timestamp
}

interface BlockedActionRecord {
  requestId: string;
  finalState: "HOLD" | "REJECT";
  reasonCodes: ReasonCode[];
  checkedInvariants: InvariantCode[];
  recordedAt: string; // ISO timestamp
}

```

5.8 Discriminated Union Types

Type	Values	Notes
AuthorityClass	"system" "ai" "analyst" "reviewer" "manager" "compliance_officer"	Core set, always registered. Extended via CerbaSealConfig.authorityClasses.extended.
HumanAuthorityClass	"analyst" "reviewer" "manager" "compliance_officer"	Subset of AuthorityClass valid in ApprovalArtifact.approverAuthorityClass.
WorkflowClass	"fraud_triage" "transaction_escalation" "account_hold_recommendation"	"fraud_triage" is hardcoded in WORKFLOWS_REQUIRING_APPROVAL
ActionClass	"allow" "hold" "reject" "escalate" "account_hold"	Registered in ALLOWED_ACTION_CLASSES Set.
DecisionState	"ALLOW" "HOLD" "REJECT"	Terminal states. HOLD is the only recoverable state (missing approval that
ProposalSourceKind	"ai" "deterministic_rule"	INV-05 uses this in combination with actorAuthorityClass to detect AI self-
UnknownableActionClass	ActionClass (string & {})	Allows any string at the boundary (JSON deserialization). assertKnownActionClass() narrows it to ActionClass or throws INV-11.

Reason Code Reference

17 reason codes are defined in `src/domain/constants/reason-codes.ts`. They appear in `DecisionTrace.reasonCodes` and `BlockedActionRecord.reasonCodes`. Multiple codes may be present per decision — the specific violation code is always accompanied by a terminal decision code.

Reason Code	Category	Produced by	Meaning
DECISION_ALLOWED	Terminal	ALLOW path	All invariants and policy rules passed. ReleaseAuthorization issued.
DECISION_HELD	Terminal	HOLD path	Request valid but approval required and absent. Resumable — approval can be obtained.
DECISION_REJECTED	Terminal	REJECT path	An invariant or policy rule failed. Terminal — no release path without a new request.
NO_POLICY_PACK	INV-01	<code>assertPolicyPack()</code>	<code>policyPackRef</code> is null.
NO_PROVENANCE	INV-02	<code>assertProvenance()</code>	<code>provenanceRef</code> is null or one of its three sub-fields is empty.
REQUIRED_APPROVAL_MISSING	INV-03	<code>assertApprovalState()</code>	Approval effectively required but <code>approvalArtifact</code> is null !' HOLD.
INVALID_APPROVAL_AUTHORITY	INV-03	<code>assertApprovalState()</code>	<code>forRequestId</code> mismatch, <code>approvedAt</code> predates <code>createdAt</code> . or approver class invalid or not in chain.
PRIVILEGED_AUTH_NOT_SATISFIED	INV-03	<code>assertApprovalState()</code>	<code>privilegedAuthSatisfied</code> is false on the <code>ApprovalArtifact</code> .
APPROVAL_SIGNATURE_MISSING	INV-03	<code>assertApprovalState()</code>	<code>immutableSignature</code> is empty or whitespace.
LOGGING_NOT_READY	INV-04	<code>assertLoggingReady()</code>	<code>loggingReady</code> is false.
AI_CANNOT_AUTHORIZE	INV-05	<code>assertProposalBoundary()</code>	<code>proposal.authorityBearing</code> is true, OR <code>proposal.sourceKind</code> "ai" with <code>actorAuthorityClass</code> "ai".
PROHIBITED_USE	INV-10	<code>assertProhibitedUse()</code>	<code>prohibitedUse</code> is true — unconditional block.
CONTROL_STALE_OR_INVALID	INV-08	<code>assertControlStatus()</code>	<code>sensitive</code> : true and <code>controlStatus.stale</code> is true or <code>criticalControlsValid</code> is false.
TRUST_STATE_INVALID	INV-09	<code>assertTrustState()</code>	<code>trustState.trusted</code> is false.
UNKNOWN_ACTION_CLASS	INV-11	<code>assertKnownActionClass()</code>	<code>proposedActionClass</code> or <code>proposal.requestedActionClass</code> not in
MALFORMED_REQUEST	INV-11 / Fallback	<code>assertRequestShape()</code> / catch	Schema validation failure. Also used by <code>assertIsGateIssued()</code> fallback and non-
INVALID_PROPOSAL	INV-12	<code>evaluate()</code> inline check	<code>proposedActionClass !== proposal.requestedActionClass</code> .

Configuration System

7.1 CerbaSealConfig & GateConfig

CerbaSealConfig registers extended authority, workflow, and action class identifiers. The core sets are hardcoded in constants and cannot be removed. Extended classes are additive. GateConfig is a discriminated union of CerbaSealConfig and the older ExecutionGateConfig (which uses additionalAuthorityClasses instead of the structured class extension format). Both are accepted by the constructor via buildAllowedAuthorityClasses().

src/config/cerbaseal-config.ts

```
// Three core sets (always registered, cannot be removed):
CORE_AUTHORITY_CLASSES = ["system", "ai", "analyst", "reviewer", "manager", "compliance_officer"]
CORE_WORKFLOW_CLASSES  = ["fraud_triage", "transaction_escalation", "account_hold_recommendation"]
CORE_ACTION_CLASSES    = ["allow", "hold", "reject", "escalate", "account_hold"]

interface CerbaSealClassExtension {
  core?: readonly string[]; // documentation only; gate ignores at runtime
  extended: readonly string[]; // additional identifiers the gate will accept
}
interface CerbaSealConfig {
  authorityClasses?: CerbaSealClassExtension;
  workflowClasses?: CerbaSealClassExtension;
  actionClasses?: CerbaSealClassExtension;
}
interface ExecutionGateConfig {
  additionalAuthorityClasses?: readonly string[];
}
type GateConfig = CerbaSealConfig | ExecutionGateConfig;
```

cerbaseal.config.json example

```
// cerbaseal.config.json – example
{
  "authorityClasses": {
    "core": ["system", "ai", "analyst", "reviewer", "manager", "compliance_officer"],
    "extended": ["risk_officer", "senior_analyst", "director"]
  },
  "workflowClasses": {
    "core": ["fraud_triage", "transaction_escalation", "account_hold_recommendation"],
    "extended": ["insurance_claim_review", "kyc_escalation"]
  }
}
```

loadCerbaSealConfig(configPath?) reads cerbaseal.config.json from the repo root (resolved via import.meta.url) or the explicit path. Returns {} (empty config = core classes only) if the file does not exist. Throws if the file exists but cannot be parsed.

7.2 CerbaSealPolicy

CerbaSealPolicy carries four categories of client-specific rules. All are optional — an absent field means that category of policy is inactive. Policy rules are additive: they can narrow permissions but cannot bypass invariants.


```
interface CerbaSealPolicy {
  actorMappings?: Record<string, AuthorityClass>;
  // e.g. { "Head of Risk": "manager", "Senior Fraud Analyst": "analyst" }

  approvalChains?: Record<string, string[]>;
  // e.g. { "fraud_triage": ["manager","compliance_officer"] }
  // Also triggers approval requirement for the listed workflow

  actionPolicies?: Record<string, Record<string, PolicyActionBehavior>>;
  // e.g. { "fraud_triage": { "allow": "requires_approval", "reject": "auto_allow" } }

  workflowRules?: WorkflowRule[];
  // e.g. [{ workflowClass: "kyc_escalation", requiresApproval: true }]
}
```

7.3 WorkflowRule & PolicyActionBehavior

Type / Field	Values / Type	Semantics
PolicyActionBehavior	"requires_approval" "auto_allow" "blocked"	"blocked" != REJECT immediately (stage 2). "requires_approval" != adds to effective approval
WorkflowRule.workflowClass	string	The workflow class identifier this rule applies to. Must match the workflowClass field on GovernedRequest.
WorkflowRule.requiresApproval	boolean	true != gate enforces approval even if caller set approvalRequired: false. false != documents intent; does NOT suppress an approvalChains requirement for the same workflow

Approval requirement is ADDITIVE. The gate requires approval if ANY of these are true: (1) request.approvalRequired === true, (2) workflowClass in WORKFLOWS_REQUIRING_APPROVAL ("fraud_triage"), (3) a workflowRules entry has requiresApproval: true, (4) workflow appears in approvalChains, (5) action has "requires_approval" policy. The most restrictive source always wins.

8

Audit & Evidence Chain

8.1 `IAuditLogService` Interface

src/services/audit/append-only-log-service.ts + src/domain/types/audit.ts

```
interface IAuditLogService {
  append(event: AuditEventPayload): AuditLogEntry;
  list(): AuditLogEntry[];
  listByRequestId(requestId: string): AuditLogEntry[];
  verifyChain(): boolean; // true iff every SHA-256 hash link is valid
}

interface AuditEventPayload {
  requestId: string;
  eventType: AuditEventType;
  payload: Record<string, unknown>;
}

interface AuditLogEntry {
  eventId: string; // UUID
  requestId: string;
  eventType: AuditEventType;
  timestamp: string;
  payloadHash: string; // SHA-256 of event payload
  previousHash: string | null; // null for first entry
  entryHash: string; // SHA-256 of (payloadHash + previousHash + metadata)
}
```

8.2 `AppendOnlyLogService` (In-Memory)

In-memory implementation. Entries are held in a private `AuditLogEntry[]` array. `append()` calls `buildEntry()` (from `audit-hash-utils.ts`) which computes the SHA-256 chain link, pushes the entry, and returns a `deepClone`. `verifyChain()` traverses all entries and validates every hash link. All returned values are `deepClone` copies — callers cannot mutate internal state.

Use `AppendOnlyLogService` in development and tests. Use `FileBackedAppendOnlyLogService` in any client deployment where audit entries must survive process restarts.

8.3 `FileBackedAppendOnlyLogService` (Persistent)

Persistent implementation backed by a JSONL file. Constructor behavior: if the file exists, reads all lines and reconstitutes the in-memory chain. If the chain is broken on disk, `verifyChain()` returns false from the first call. `append()` calls `buildEntry()` then `appendFileSync()` — the entry is on disk before `append()` returns (crash-safe). `deepClone` applied to all returned values.

Property	Detail
File format	JSONL — one JSON-serialised AuditLogEntry per line. Human-readable and grep-friendly.
Write durability	appendFileSync() — synchronous write to disk. Process crash after append() returns means the entry is durably recorded.
Constructor	constructor(filePath: string) — reads and reconstitutes the chain on startup. No options; the path is the only parameter.
Chain loading	Each line is JSON.parse()d to AuditLogEntry and pushed to the in-memory array. Hashes are stored in the JSONL, not recomputed.
Verification	verifyChain() delegates to verifyChainUtil from audit-hash-utils.ts — same algorithm as in-memory, same guarantee.
Production setup	Set LOG_PATH env var to a persistent volume path (e.g. /app/data/audit.jsonl). Docker Compose mounts ./data:/app/data.

8.4 EvidenceBundleService

EvidenceBundleService.createBundle({ request, gateResult }) assembles an EvidenceBundle. It first calls assertIsGateIssued(gateResult) — any result not in _gateIssuedResults throws CerbaSealError(INV-06) before any log append occurs. It then appends multiple audit events and returns the complete bundle.

Audit event appended	AuditEventType	Condition
Request evaluated	"REQUEST_EVALUATED"	Always — first event; includes finalState, workflowClass, evidenceBundleId, actorId in payload
Release authorized	"RELEASE_AUTHORIZED"	When finalState === "ALLOW"
Action blocked	"ACTION_BLOCKED"	When finalState === "HOLD" or "REJECT"
Evidence bundle created	"EVIDENCE_BUNDLE_CREATED"	After full bundle assembly

The returned EvidenceBundle contains: evidenceBundleId ("evidence_" + requestId), the original GovernedRequest (deepClone), the DecisionEnvelope, ReleaseAuthorization or null, BlockedActionRecord or null, eventChain (all AuditLogEntry objects for this requestId), and createdAt.

8.5 Audit Event Types

src/domain/types/audit.ts — AuditEventType

```
type AuditEventType =
  | "REQUEST_EVALUATED"      // gate.evaluate() called
  | "RELEASE_AUTHORIZED"    // ALLOW decision — action may proceed
  | "ACTION_BLOCKED"        // HOLD or REJECT — action blocked
  | "EVIDENCE_BUNDLE_CREATED" // EvidenceBundleService.createBundle() completed
  | "EXPORT_MANIFEST_CREATED" // pnpm export:proof completed
```


HTTP API Reference

The deployment-starter/index.ts HTTP server exposes the enforcement gate as a minimal REST API. The server is unauthenticated — network-level access control is the operator's responsibility. Both endpoints always return JSON. The server is implemented with Node.js core `http.createServer()` (no Express or other framework).

POST /evaluate

Property	Detail
Method + Path	POST /evaluate
Content-Type	application/json
Request body	JSON-serialised GovernedRequest object (see §5.1 for all 17 fields)
Success (200)	JSON-serialised GateResult { decisionEnvelope, releaseAuthorization, blockedActionRecord }
Body parse error (400)	{ "error": "Invalid JSON — expected a GovernedRequest object" }
Read error (400)	{ "error": "Failed to read request body" }
Gate exception (422)	{ "error": "<CerbaSealError.message>" } — always a controlled message, never a raw stack trace
Side effect	logService.append({ requestId, eventType: "REQUEST_EVALUATED", payload: { finalState, workflowClass, evidenceBundleId, actorId } })
Not found (404)	{ "error": "Not found", "routes": ["POST /evaluate", "GET /health"] }


```
# ALLOW scenario – all invariants satisfied, manager approval present
curl -X POST http://localhost:3000/evaluate \
-H "Content-Type: application/json" \
-d '{
  "requestId": "req-001",
  "workflowClass": "fraud_triage",
  "jurisdiction": "US-CA",
  "actorId": "analyst-jane",
  "actorAuthorityClass": "analyst",
  "proposedActionClass": "allow",
  "proposal": {
    "proposalSourceKind": "ai",
    "authorityBearing": false,
    "requestedActionClass": "allow",
    "confidence": 0.94,
    "reasonCodes": ["MODEL_HIGH_CONF"],
    "proposalCreatedAt": "2026-06-08T10:00:00Z"
  },
  "sensitive": true,
  "prohibitedUse": false,
  "policyPackRef": { "id": "acme-policy", "version": "1.0.0" },
  "provenanceRef": { "modelVersion": "v3.1", "ruleSetVersion": "v2.0", "sourceHash":
"sha256:abc" },
  "approvalRequired": true,
  "approvalArtifact": {
    "approvalId": "appr-001", "approverId": "mgr-john",
    "forRequestId": "req-001",
    "approverAuthorityClass": "manager",
    "privilegedAuthSatisfied": true,
    "immutableSignature": "sig-abc",
    "approvedAt": "2026-06-08T10:01:00Z"
  },
  "loggingReady": true,
  "controlStatus": { "criticalControlsValid": true, "stale": false, "verificationRunId": "run-1" },
  "trustState": { "trusted": true, "trustStateId": "ts-001" },
  "createdAt": "2026-06-08T10:00:00Z"
}'
```

GET /health

Property	Detail
Method + Path	GET /health
Response	200 OK always (degraded state is signalled via status field, not HTTP code)
Response body	{ "status": "ok" "degraded", "gateReady": true, "auditChainValid": boolean, "uptime": number }
status	"ok" when auditChainValid true; "degraded" when verifyChain() returns false (chain tampered or disk corrupted)
gateReady	Always true if server started cleanly. Gate service loaded config and policy at module init time.
auditChainValid	logService.verifyChain() — checks the full SHA-256 chain. "degraded" means the log has been tampered with.
uptime	(Date.now() - START_MS) / 1000 — seconds since server start


```
$ curl http://localhost:3000/health
{
  "status": "ok",
  "gateReady": true,
  "auditChainValid": true,
  "uptime": 142.7
}
```

Environment Variables

Variable	Default	Purpose
PORT	3000	TCP port the HTTP server binds to. Replit assigns PORT automatically; Docker Compose maps it from .env.
LOG_PATH	"memory"	"memory" != AppendOnlyLogService (in-process, not persistent). Any other value != FileBackedAppendOnlyLogService at that path.
CONFIG_PATH	<cwd>/cerbaseal.config.json	Explicit path to config file. join(process.cwd(), "cerbaseal.config.json") when not set.
POLICY_PATH	<cwd>/cerbaseal.policy.json	Explicit path to policy file. join(process.cwd(), "cerbaseal.policy.json") when not set.

Deployment Architecture

10.1 Path A — Docker Compose (Recommended)

Docker Compose is the recommended deployment path. It requires Docker "e 20 and Docker Compose "e 2.0 on the target machine — no Node.js, pnpm, or TypeScript toolchain needed. The enforcement stack runs inside a self-contained container.

Property	Detail
Base image (both stages)	node:22-alpine
Build stage	Install pnpm@9 globally. Run pnpm install in deployment-starter/. Resolve the file:../ cerbaseal-review symlink to concrete files (cp -r) so the runtime image contains no symlinks.
Runtime command	node_modules/.bin/tsx deployment-starter/index.ts from WORKDIR /app
Audit log volume	./data:/app/data — audit.jsonl persists across container restarts. Set LOG_PATH=/app/data/audit.jsonl in .env.
Health check directive	wget -qO- http://localhost:3000/health — 30s interval, 5s timeout, 3 retries, 10s start period
Config/policy override	Mount a custom file: -v ./client-policy.json:/app/cerbaseal.policy.json:ro
Restart policy	restart: unless-stopped — container recovers from crashes without operator intervention
Port mapping	"3000:3000" by default in docker-compose.yml. Change via PORT env var and port mapping together.

Docker Compose quick start

```
# From the deployment-starter/ directory:
cp .env.template .env           # copy template
# Edit .env: CERBASEAL_ENV=production, LOG_PATH=/app/data/audit.jsonl
docker compose up                # builds image (~3-5 min first time)

# Confirm healthy:
curl http://localhost:3000/health
# { "status": "ok", "gateReady": true, "auditChainValid": true, "uptime": 1.4 }
```

10.2 Path B — Node.js Direct

Used when the client environment has Node.js 18+ and pnpm available, or when the partner needs full source access during pilot configuration. This is also the path used by the Replit deployment starter.

Step	Command	Expected Outcome
1	pnpm install	node_modules populated; pnpm-lock.yaml unchanged
2	pnpm test	391 tests passing across 17 test files (all must pass)
3	pnpm audit:repo	16/16 automated governance checks passing
4	pnpm setup	Interactive wizard writes cerbaseal.config.json and cerbaseal.policy.json
5	tsx deployment-starter/verify.ts	9/9 assertions passing across 3 scenarios (ALLOW, HOLD, REJECT)
6	pnpm export:proof	cerbaseal-proof.json written with checksum and git provenance
7	pnpm verify:proof	Checksum match confirmed — proof snapshot integrity verified
8	tsx deployment-starter/index.ts	Server listening — logs port, log path, config path, policy path

Dependency model: deployment-starter/package.json declares "cerbaseal-review": "file:../" — a pnpm file: reference to the repo root. Running pnpm install inside deployment-starter/ creates a node_modules/cerbaseal-review symlink. When published to npm in v0.2.0+, this changes to "cerbaseal-review": "^0.2.0". All import paths in deployment-starter/index.ts remain identical — only the package.json line changes.

Test Coverage

The test suite covers enforcement gate invariants, audit chain integrity, evidence assembly, configuration and policy layers, adversarial bypass attempts, integration scenarios, and regression snapshots. All tests run with Vitest. The suite must pass 391/391 before any client deployment.

Test File	Category	Tests	Coverage Focus
<code>execution-gate-service.test.ts</code>	Unit	19	All 12 invariants, policy rules, ALLOW/HOLD/REJECT decision states. GateResult structure. actor mapping
<code>adversarial-integrity.test.ts</code>	Security	66	Tamper attempts, hand-constructed GateResult, invariant circumvention. approval forger patterns
<code>security/contextual-boundary.test.ts</code>	Security	25	Authority class boundary isolation, cross-context access attempts. role escalation vectors
<code>security/fail-closed.test.ts</code>	Security	2	Gate fails closed (REJECT) on schema error and on any unknown / unexpected input
<code>security/non-forgery.test.ts</code>	Security	2	SHA-256 chain non-forgery — tampered AuditLogEntry detected by <code>verifyChain()</code>
<code>security/misuse-scenarios.test.ts</code>	Security	34	Real-world misuse: AI self-auth, stale controls on sensitive, unsigned approval. prohibited use override
<code>audit-evidence-export.test.ts</code>	Integration	6	EvidenceBundle assembly, ExportManifest structure, <code>export:proof</code> + <code>verify:proof</code> round-trip
<code>persistent-audit-log.test.ts</code>	Integration	12	FileBackedAppendOnlyLogService: crash recovery, JSONL persistence. chain reconstitution
<code>diagnostic-report-service.test.ts</code>	Integration	5	DiagnosticReport assembly, severity classification (CRITICAL/HIGH/MEDIUM/LOW). replav consistency
<code>config-policy-layer.test.ts</code>	Unit	—	CerbaSealConfig loading, class extension, CerbaSealPolicy validation. all four policy categories
<code>integration/full-flow.test.ts</code>	Integration	1	End-to-end: GovernedRequest !' gate !' audit !' evidence !' export !' <code>verify</code> !' proof match
<code>integration/browser-demo-routes.test.ts</code>	Integration	28	Browser demo HTTP server routes: all API endpoints, portal pages. and navigation
<code>integration/review-portal-routes.test.ts</code>	Integration	110	Review portal routes: decision summary, evidence retrieval, replav API. diagnostic API
<code>integration/external-signal-examples.test.ts</code>	Integration	16	External actor integration: multi-tenant scenarios, role mapping. chain <code>verify</code> via HTTP
<code>integration/support-readiness.test.ts</code>	Integration	23	Support scenarios: operator action reports, system health checks. all three decision states
<code>integration/system-integration.test.ts</code>	Integration	1	Full system integration: gate + policy + audit + evidence all working together end-to-end
<code>snapshots/enforcement-loop.snapshot.test.ts</code>	Snapshot	41	Regression snapshots: specific GovernedRequest shapes always produce identical GateResult outputs

pnpm audit:repo — 16 Automated Governance Checks

The repo audit is a separate layer from the test suite. It verifies governance integrity properties of the repository itself, not just source behaviour. All 16 checks must pass before any client deployment. Check #1 requires the test suite to pass, so a failing test suite blocks all audit checks.

#	What It Verifies
01	Test suite passes (391/391)
02	No prohibited cross-layer imports in enforcement core (gate cannot import from evidence layer)
03	Exactly 12 invariant codes defined in INVARIANTS constant
04	Every InvariantCode referenced in tests is present in INVARIANTS
05	Every ReasonCode referenced in tests is present in REASON_CODES
06	ExecutionGateService does not export _gateIssuedResults or internal WeakSet
07	IAuditLogService interface is satisfied by both AppendOnlyLogService and FileBackedAppendOnlyLogService
08	FileBackedAppendOnlyLogService uses appendFileSync (not async writeFile)
09	DecisionEnvelope.immutable field is literal type true in the type definition
10	Tests import via public API surface only (no direct internal module coupling)
11	cerbaseal.config.json parses without error and matches CerbaSealConfig schema
12	cerbaseal.policy.json parses without error and matches CerbaSealPolicy schema
13	CerbaSealError always carries invariant code, reason code, and finalState on every constructor call
14	No stale test-count references in documentation files
15	Partner kit documents exist and contain required section headings
16	export:proof + verify:proof round-trip produces identical checksums (idempotent)

Partner Delivery Framework

Partner Kit Documents (docs/partner-kit/)

File	Title	Primary Audience	When to Use
00-OVERVIEW.md	Partner Delivery Kit Overview	All partners	Kit navigation, cert levels, tools, Tier 3 escalation boundaries
01-sales-brief.md	Sales Brief	Partner sales leads	ICP qualification, positioning, commercial model. opening
02-technical-brief.md	Technical Brief	Partner tech leads	7-layer architecture, invariants, evidence chain. integration patterns
03-deployment-guide.md	Deployment Guide	Partner engineers	Step-by-step: env setup, wizard, audit. verification. evidence export
04-pilot-guide.md	1-Day Pilot Guide	Pilot leads	CerbaSeal Express Pilot — morning to evidence package in one day
05-support-guide.md	Support Guide	All partners	10 most common issues with root cause. diagnosis steps. resolution
06-objection-handling.md	Objection Handling	Partner sales leads	8 enterprise objections with CTO / CISO / CPO / CFO calibrated
07-certification-framework.md	Certification Framework	All partners	3 cert levels: competencies, practical exercises, pass/fail criteria

Certification Levels

Level	Name	Prerequisite	What a Pass Proves
Level 1	Deploy	None	Clone repo, run pnpm install, pass pnpm test (391/391), pass pnpm audit:repo (16/16), run 3-scenario verify (9/9 assertions), export and verify
Level 2	Configure	Level 1	Author a complete cerbaseal.policy.json for a new client workflow without assistance. All approval chains, actor mappings, action policies correct.
Level 3	Lead Pilots	Level 2 + one client delivery	Run a full pilot independently from kickoff to signed evidence package. Deliver all 7 audit-kit documents. Resolve support issues without Tier 3 escalation.

Engagement Lifecycle

Phase	Who	Deliverable
Pre-sales	Partner sales lead	Qualify client via ICP scorecard. Technical discovery meeting. Sales Brief objection handling.
Scoping	Partner tech lead	Workflow mapping: governed workflow, AI actor, human approver roles, action classes, iurisdiction.
Deployment	Partner engineer	Path A or B. pnpm test 391/391, pnpm audit:repo 16/16. Config and policy files written.
Configuration	Partner engineer	cerbaseal.policy.json authored: approval chains, actor mappings, action policies per client workflow.
Verification	Partner engineer	3-scenario verify.ts run. 9/9 assertions pass. Results reviewed with client.
Pilot run	Partner + client	Real or representative requests submitted through gate. Evidence captured automatically by audit log.
Evidence close	Partner	pnpm export:proof !' cerbaseal-proof.json. pnpm verify:proof !' checksum confirmed. Delivered to client.
Handover	Partner	Client operations team trained on diagnostic portal, support guide, and pnpm audit:repo.

What Requires Lamont Labs Tier 3 Escalation

Level 2+ partners are expected to resolve 80%+ of issues independently. The following items require direct Lamont Labs involvement:

- New core enforcement invariants — adding a new hard rule requires TypeScript changes and new tests.
- New ProposalSourceKind values beyond "ai" and "deterministic_rule".
- PKI-backed cryptographic signing of DecisionEnvelope objects (HMAC is self-service; PKI requires implementation work).
- Third-party security review coordination — not completed for v0.1.0; scheduled for v0.2.0.
- Windows deployment — not tested in v0.1.0; support posture is unclear.

Security Properties & Threat Model

Core Security Properties

Property	Mechanism	Test Coverage
No execution without gate evaluation	_gateIssuedResults WeakSet + assertIsGateIssued() in EvidenceBundleService. Any GateResult not from	adversarial-integrity.test.ts, misuse-scenarios.test.ts
AI cannot self-authorise	INV-05 checked unconditionally. "ai" actorAuthorityClass blocked as approver; AI-sourced proposal from AI actor	INV-05, execution-gate-service.test.ts
Tamper-evident audit log	SHA-256 forward chaining. Every AuditLogEntry.entryHash covers previousHash.	security/non-forgery.test.ts
Immutable decision envelopes	DecisionEnvelope.immutable: true is a literal type. WeakSet registration ties the envelope to a specific	INV-07, adversarial-integrity.test.ts
Prohibited use unconditional block	INV-10 fires after INV-11/12 but before all other invariants. prohibitedUse: true != REJECT regardless of	INV-10, misuse-scenarios.test.ts
Fail-closed on all unknown input	Unknown action class != CerbaSealError throw (REJECT). Schema error != controlled REJECT. Non-	security/fail-closed.test.ts
Policy cannot relax invariants	Policy stages 1–4 are evaluated around the invariant sequence. No policy stage can suppress or skip an	adversarial-integrity.test.ts
Approval non-reuse	INV-03: approvalArtifact.forRequestId must equal request.requestId. A valid approval from request A	assertApprovalState(), adversarial-integrity.test.ts
Approval timestamp integrity	INV-03: approvedAt must be a valid ISO date and must not predate request.createdAt. Prevents forged or	assertApprovalState(), adversarial-integrity.test.ts
No network dependency at evaluation	evaluate() is a pure synchronous function. No network calls, no file reads, no side effects. Gate availability is independent of external service availability.	execution-gate-service.test.ts

Threat Model — In Scope for v0.1.0

Threat	Attacker Goal	Mitigation in v0.1.0
AI self-authorisation	AI system approves its own proposal, bypassing human review	INV-05: actorAuthorityClass "ai" structurally blocked as approver; proposalSourceKind "ai" + actorClass "ai" always REJECT
Approval forgery / replay	Attacker reuses a valid ApprovalArtifact across different requests	INV-03: forRequestId must match requestId; approvedAt must not predate createdAt
Audit log tampering	Attacker modifies historical entries to hide a blocked decision	SHA-256 forward chain; verifyChain() reported on GET /health
Gate bypass via evidence	Caller hand-constructs GateResult, skips gate.evaluate(), feeds directly to	assertIsGateIssued() + _gateIssuedResults WeakSet in EvidenceBundleService
Policy relaxation	Partner misconfigures policy to exempt a workflow from invariant checks	Policy layer evaluated after invariants; no policy mechanism can suppress INV-01 through INV-12
Stale control exploitation	Caller submits sensitive request after controls have expired	INV-08: controlStatus.stale must be false and criticalControlsValid must be true for sensitive requests
Prohibited use bypass	Caller submits prohibited use request with all other fields valid	INV-10: prohibitedUse checked early in evaluation sequence — unconditional REJECT regardless of approval or policy
AI authority boundary	AI sets authorityBearing: true on its proposal to claim execution authority	INV-05: proposal.authorityBearing === true always REJECT, regardless of actorAuthorityClass

Out of Scope for v0.1.0

- PKI-backed cryptographic signing of DecisionEnvelope objects (HMAC available; PKI planned v0.2.0).
- Third-party penetration testing or independent security audit (scheduled for v0.2.0).
- Network-level access control and authentication of HTTP API callers (operator responsibility).
- Denial-of-service resilience — rate limiting, circuit breaking, WAF (operator responsibility).
- Key management infrastructure for approval signatures (v0.1.0 checks presence, not cryptographic validity).

Security review status: CerbaSeal-Core v0.1.0 has not undergone a formal third-party security review. Partners must communicate this to clients during pilot kickoff. An independent security assessment is a milestone gate for v0.2.0.

CerbaSeal-Core

v0.1.0 · System Architecture & Technical Reference

This document is the authoritative technical reference for CerbaSeal-Core.
Issued under the Lamont Labs Partner Confidentiality Agreement.
Redistribution outside the authorised partner organisation is prohibited.

